

UNIVERSIDAD NACIONAL DE CUYO

Facultad de Ingeniería

Licenciatura en Ciencias de la Computación

Ingeniería del Software II

Trabajo Práctico Integrador

Parte N°1

Sistema E-Commerce para la tienda de ropa deportiva «Zero»



Integrantes:

- Maximo Perez Beltramone
- Manuel Gomez Montero
- Valentin Emanuel Motos
- Diego Cazon Condori

Profesor:

- Lic. Leandro Spadaro

Año:

- 2026

Índice

1. Introducción	4
1.1. Objetivo	4
1.2. Descripción del sistema	4
1.3. Tecnologías utilizadas	4
2. Proceso de desarrollo	5
2.1. Proceso Unificado de Desarrollo (RUP)	5
2.2. Organización del equipo	5
2.3. Pruebas de software	5
3. Arquitectura y diseño del software	6
3.1. Arquitectura en capas	6
3.2. Diagrama de paquetes	6
4. Diagrama de clases de diseño	8
5. Diagrama Entidad-Relación de la base de datos	10
6. Requerimientos no funcionales	12
6.1. Seguridad	12
6.2. Usabilidad e interfaz	12
6.3. Rendimiento y disponibilidad	13
6.4. Confiabilidad e integridad de datos	13
6.5. Mantenibilidad y calidad	13
6.6. Portabilidad, instalación y tecnología	14
6.7. Restricciones del negocio	14
7. Patrones de software	15
7.1. Capas	15
7.2. Modelo-Vista-Controlador (MVC)	15
7.3. DAO (Data Access Object)	15
7.4. OID (Object Identifier)	16
7.5. DTO (Data Transfer Object)	16
7.6. Patrones GRASP aplicados	16

8. Casos de uso críticos	17
8.1. CU01 · Generar orden de compra	18
8.2. CU02 · Generar factura	23
8.3. CU03 · Actualizar stock	27
9. Propuesta de mejora	33
9.1. Investigación de mercado	33
9.2. Funcionalidad propuesta: calendario de llegadas y envíos	33
9.3. Cambios en el diseño	33
9.4. Diagrama de clases de diseño de la propuesta	35
10. Conclusiones	36
Anexo A. Ubicación de los entregables	36

1. Introducción

1.1. Objetivo

El presente informe documenta el análisis, el diseño y la implementación del sistema solicitado en el Trabajo Práctico Integrador Parte N°1 de la asignatura Ingeniería del Software II. El objetivo es desarrollar un software de tipo web con arquitectura Cliente-Servidor, de al menos tres capas, aplicando patrones de software y siguiendo el Proceso Unificado de Desarrollo (RUP).

1.2. Descripción del sistema

El equipo fue contratado por la tienda de ropa deportiva unisex «Zero», de la provincia de Mendoza, para desarrollar un sistema de comercio electrónico. La tienda organiza sus productos en las categorías Niños, Niñas, Mujeres y Hombres, cada una dividida en Ropa, Calzado y Accesorios. El sistema cubre:

- Un sitio institucional con el catálogo de productos con stock, el detalle de cada producto y una sección de ofertas.
- Compras en línea mediante un carrito, con distintas formas de pago (efectivo, transferencia y Mercado Pago simulado), confirmación por correo electrónico y seguimiento del estado de cada compra desde el perfil del cliente.
- La administración de productos, precios con vigencias (actualizados periódicamente por la inflación), proveedores, compras a proveedores y stock.
- Usuarios con roles (Jefe, Administrativo y Cliente), contraseñas encriptadas y activación de cuenta por correo.
- Un newsletter con los productos en oferta enviado cada 10 días.
- Reportes de ventas, de stock (con aviso al proveedor por WhatsApp) y de proveedores.

1.3. Tecnologías utilizadas

Aspecto	Tecnología
Lenguaje y marco de trabajo	Java 21 y Spring Boot 3.5
Vista	Thymeleaf (Layout Dialect y Extras Spring Security), HTML, CSS y Bootstrap 4 (template adaptado a la marca)
Persistencia	Spring Data JPA con Hibernate
Base de datos	SQLite embebida
Seguridad	Spring Security con contraseñas BCrypt
Correo	Spring Mail sobre un servidor SMTP configurable
Servidor de aplicaciones	Tomcat embebido
Pruebas	JUnit 5, Mockito, Spring Test y GreenMail
Versionado y organización	Git (GitHub) y tablero Kanban en Trello

2. Proceso de desarrollo

2.1. Proceso Unificado de Desarrollo (RUP)

El proyecto se desarrolló de forma iterativa e incremental. Cada etapa del desarrollo constituyó una iteración con un conjunto de funcionalidades completas (vista, controlador, servicio y pruebas). Las etapas se asociaron a las fases de RUP de la siguiente manera:

Fase	Iteraciones	Actividades principales
Iniciación	Preliminar	Análisis del enunciado, identificación de los requerimientos funcionales y no funcionales, estimación.
Elaboración	Etapa 0	Definición de la arquitectura en capas, entidades JPA y repositorios del diagrama de clases, integración del template y kit de componentes.
Construcción	Etapas 1 a 5	Servicios transversales (seguridad, ubicación, categorías, correo); productos, precios y cuentas; abastecimiento y catálogo; venta y postventa; reportes y dashboard.
Transición	Etapa 6	Pruebas de integración, documentación final y presentación.

2.2. Organización del equipo

Las tareas se organizaron en un tablero Kanban de Trello con las columnas Pendiente, En progreso, Probada y Finalizada. Cada tarea se desarrolló en una rama propia del repositorio Git y se integró mediante un Pull Request revisado y probado por otro integrante del equipo. Al cierre de cada etapa se probó el flujo completo en conjunto.

2.3. Pruebas de software

El sistema cuenta con 72 clases de prueba y 379 casos automatizados (pruebas unitarias con JUnit y Mockito, y pruebas de integración con Spring Test y GreenMail como servidor de correo en memoria). Se ejecutan con el comando *mvnw clean verify*.

3. Arquitectura y diseño del software

3.1. Arquitectura en capas

El sistema es una aplicación web Cliente-Servidor organizada en tres capas según el patrón Modelo-Vista-Controlador:

Capa	Paquete	Responsabilidad
Presentación	controllers y templates	Recibe las peticiones HTTP, invoca a los servicios y elige la vista Thymeleaf. No contiene reglas de negocio.
Negocio	services	Validaciones, reglas del negocio y transacciones. No conoce la vista.
Persistencia	repositories	Acceso a la base de datos mediante repositorios Spring Data JPA.

Un controlador nunca utiliza un repositorio en forma directa: siempre lo hace a través de un servicio. Todas las entidades heredan de *BaseEntity*, que define el identificador (UUID) y la marca de baja lógica.

3.2. Diagrama de paquetes

El diagrama de paquetes muestra los paquetes Vista (82 plantillas), Controlador (38 controladores, divididos en *admin*, *cliente* y *publico*) y Modelo, que agrupa Servicio (34 clases), Repositorio (30) y Dominio (30 entidades). Las dependencias se obtuvieron de las importaciones reales del código:

- Vista ↔ Controlador: los formularios y enlaces invocan a los controladores, y estos devuelven el nombre de la plantilla.
- Controlador → Servicio y Dominio.
- Servicio → Repositorio y Dominio; Servicio → Vista, porque el servicio de correo arma los mensajes con las plantillas de *email*.
- Repositorio → Dominio.



4. Diagrama de clases de diseño

El diagrama de clases de diseño define las entidades del dominio, sus atributos, métodos y relaciones, agrupadas en: ubicación (País, Provincia, Departamento, Localidad, Dirección), personas y usuarios (Persona, Cliente, Empleado, Usuario, Nacionalidad), empresa y contactos, catálogo y precios (Categoría, Subcategoría, Producto, VigenciaPrecio, Imagen), ventas y compras (OrdenCompra, DetalleCompra, Factura, FacturaCliente, FacturaProveedor, DetalleFactura, FormaDePago, Proveedor) y stock.

Decisiones de diseño relevantes:

- **OrdenCompra** representa el carrito del cliente. Al confirmarse la compra se genera una **FacturaCliente**; la compra a un proveedor se registra como **FacturaProveedor**.
- El **stock** se guarda como movimientos: el stock actual de un producto es el saldo de su último movimiento y nunca puede quedar negativo.
- El precio vigente de un producto es la **VigenciaPrecio** sin fecha de fin; cada detalle de factura guarda el precio del momento de la venta.
- Las jerarquías **Persona**, **Contacto** y **Factura** se persisten con herencia JOINED.



5. Diagrama Entidad-Relación de la base de datos

El diagrama entidad-relación se construyó a partir de la base de datos que genera Hibernate y se verificó contra las entidades del código. Comprende 29 tablas y 35 relaciones. Cada tabla muestra su clave primaria y sus atributos, y cada relación se representa con un rombo que indica su significado (por ejemplo, «compra», «contiene», «emite»).

- **Cardinalidades:** «*» para claves foráneas comunes; «0..1» del lado de la tabla referenciada cuando el código permite que la clave foránea quede vacía (por ejemplo, la imagen de una persona o el empleado de una factura).
- **Herencia:** las tablas cliente y empleado comparten la clave primaria con persona; contacto_correo_electronico y contacto_telefonico con contacto; factura_cliente y factura_proveedor con factura (relación «es un/una»).
- **Integridad:** SQLite trabaja con las claves foráneas activas, y todas las tablas usan un identificador UUID y una columna de baja lógica.

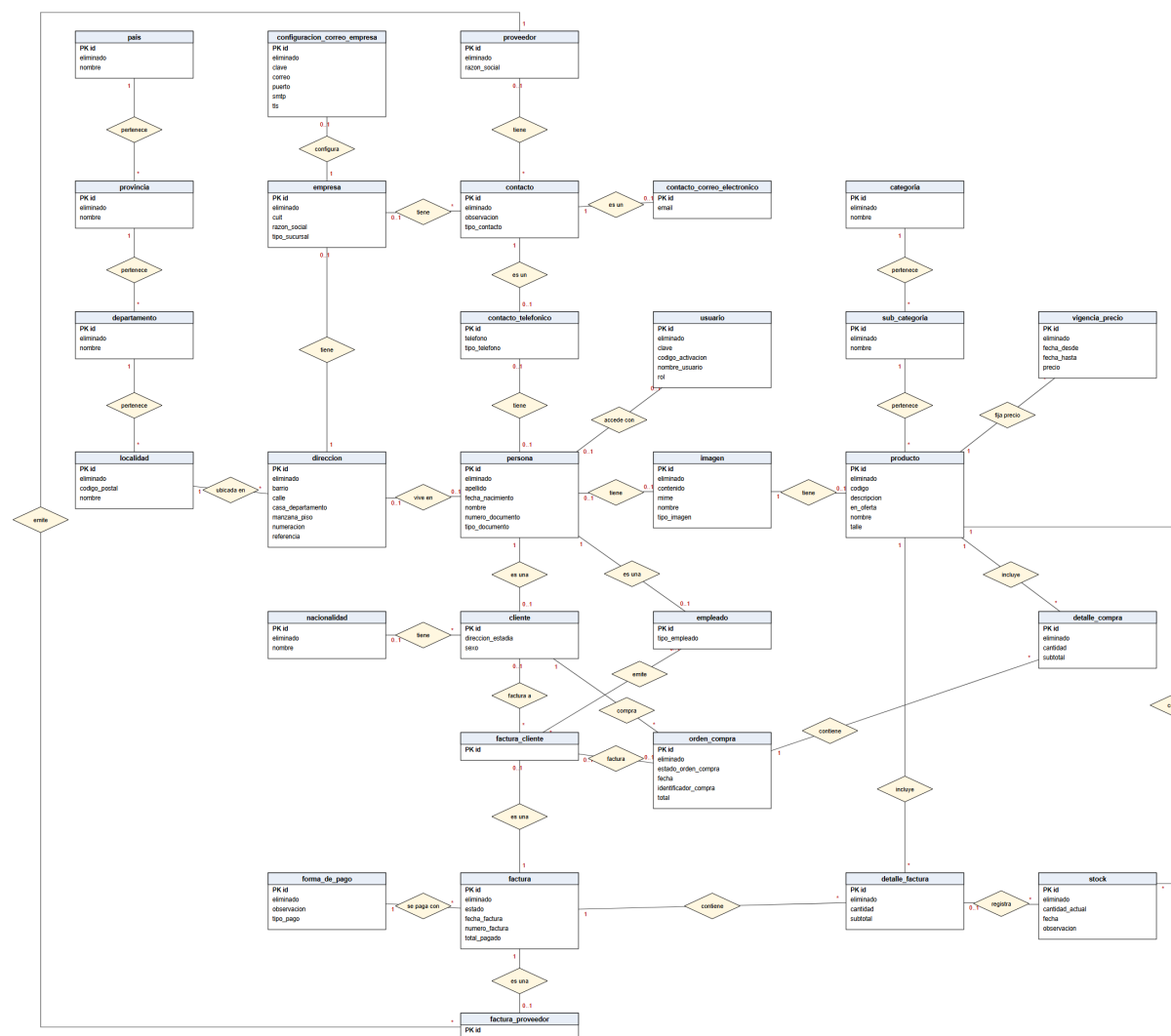


Figura 3. Diagrama entidad-relación de la base de datos.

6. Requerimientos no funcionales

Los requerimientos no funcionales describen las cualidades y restricciones del sistema. Surgen del enunciado y de las decisiones del equipo; para cada uno se indica cómo se cumple en el sistema desarrollado.

6.1. Seguridad

ID	Requerimiento	Cómo se cumple
RNF01	Las contraseñas de los usuarios se almacenan encriptadas; nunca se guardan en texto plano.	Encriptación con BCrypt (<i>BCryptPasswordEncoder</i> en <i>SecurityConfig</i>).
RNF02	El ingreso al sistema se realiza con usuario (correo personal) y contraseña. La contraseña debe tener entre 8 y 64 caracteres y confirmarse al registrarse.	Login con Spring Security (formulario <i>/login</i>). Validación en <i>UsuarioService</i> .
RNF03	Una cuenta de cliente nueva queda inactiva hasta que se ingresa el código de activación de 6 dígitos enviado por correo. Las cuentas inactivas o dadas de baja no pueden iniciar sesión.	Atributo <i>codigoActivacion</i> de <i>Usuario</i> ; excepciones <i>ActivacionPendienteException</i> y <i>CuentaEliminadaException</i> .
RNF04	El acceso a cada sección se limita según el rol: JEFE accede a todo el panel; ADMINISTRATIVO a todo el panel salvo usuarios y configuración de empresa/correo; CLIENTE a su perfil, carrito y compras. Un acceso sin permiso muestra un error 403.	Reglas por ruta en <i>SecurityConfig</i> (<i>/admin/**</i> , <i>/cliente/**</i>) y menús según el rol.
RNF05	Un cliente solo puede ver y operar sobre sus propios datos (carrito, compras y perfil), aunque modifique los identificadores en la URL.	Verificación del cliente dueño en <i>OrdenCompraService</i> y <i>CompraClienteController</i> (403 si es de otro cliente).
RNF06	Todos los formularios están protegidos contra ataques CSRF y las acciones que modifican datos se envían por POST.	Protección CSRF de Spring Security activa; token incluido en los layouts y formularios.
RNF07	Al cerrar sesión se invalida la sesión y se elimina la cookie de sesión.	Configuración de logout en <i>SecurityConfig</i> .
RNF08	Solo se aceptan imágenes JPG, PNG o WEBP de hasta 2 MB.	Validación en <i>ImagenService</i> y límite de subida en <i>application.properties</i> .

6.2. Usabilidad e interfaz

ID	Requerimiento	Cómo se cumple
RNF09	El sitio institucional usa una maqueta HTML/CSS y la gestión administrativa un panel tipo dashboard, con un único template visual (basado en Bootstrap 4) y componentes comunes.	Layouts <i>publico.html</i> y <i>admin.html</i> , kit de fragments reutilizables y estilos de marca en <i>zero.css</i> .
RNF10	El sitio público es responsive: se puede usar desde celulares, tablets y computadoras.	Grilla responsive de Bootstrap 4 y meta viewport en los layouts.
RNF11	Toda la interfaz y los mensajes están en español. Los errores de validación se muestran con mensajes claros.	<i>ErrorMessageException</i> con mensajes en español mostrados como mensajes flash.

ID	Requerimiento	Cómo se cumple
RNF1 2	Los estados de pedidos y facturas se muestran siempre con el mismo color y etiqueta en todo el sistema.	Fragment <i>badge-estado</i> único.
RNF1 3	Ante un error, el usuario ve páginas de error propias del sistema (403, 404 y 500) en lugar de errores técnicos.	Plantillas <i>error/403</i> , <i>error/404</i> y <i>error/500</i> ; <i>GlobalExceptionHandler</i> .
RNF1 4	Los listados largos tienen filtros y paginación, y los reportes se pueden exportar a CSV e imprimir.	Fragments de filtros y paginación; <i>ExportadorCsv</i> y vista de impresión.

6.3. Rendimiento y disponibilidad

ID	Requerimiento	Cómo se cumple
RNF1 5	El envío de correos no demora la operación del usuario: se realiza en segundo plano y cada intento tiene un tiempo máximo de espera de 10 segundos.	<i>@Async</i> en <i>EmailService</i> (<i>AsyncConfig</i>) y timeouts SMTP de 10 000 ms.
RNF1 6	Las tareas automáticas se ejecutan sin intervención: el newsletter de ofertas se envía cada 10 días y diariamente se detectan los productos con precio sin actualizar hace más de 2 meses (horario de Argentina).	<i>NewsletterScheduler</i> (9:00) y <i>PrecioScheduler</i> (9:15), zona <i>America/Argentina/Buenos_Aires</i> .

6.4. Confiabilidad e integridad de datos

ID	Requerimiento	Cómo se cumple
RNF1 7	No se eliminan datos físicamente: las bajas son lógicas y los registros dados de baja dejan de mostrarse.	Atributo <i>eliminado</i> en <i>BaseEntity</i> ; métodos <i>eliminarX()</i> y <i>listarXActivo()</i> .
RNF1 8	La base de datos mantiene la integridad referencial y las operaciones de negocio se ejecutan de forma transaccional (se completan enteras o no se aplican).	SQLite con <i>foreign_keys=on</i> ; services con <i>@Transactional</i> .
RNF1 9	El stock nunca puede quedar negativo y una venta conserva el precio vigente al momento de la compra, aunque el precio cambie después.	Control en <i>StockService</i> ; el precio se guarda en cada <i>DetalleFactura</i> .
RNF2 0	Cada registro se identifica con un identificador único universal (UUID).	<i>@UuidGenerator</i> en <i>BaseEntity</i> .

6.5. Mantenibilidad y calidad

ID	Requerimiento	Cómo se cumple
RNF2 1	El software es web, con arquitectura Cliente-Servidor de al menos tres capas (presentación, negocio y persistencia) siguiendo el patrón MVC. Un controlador nunca accede directamente a la persistencia.	Capas controllers → services → repositories, vistas Thymeleaf.
RNF2 2	Se aplican patrones de software: Capas, MVC, DAO, DTO (reportes), Inyección de dependencias, Polimorfismo, Creador, Experto y Alta cohesión / Bajo acoplamiento.	Ver diagrama de paquetes y diagrama de clases de diseño.

ID	Requerimiento	Cómo se cumple
RNF2 3	El código respeta convenciones comunes: nombres en español iguales al diagrama de clases, un service por entidad e inyección de dependencias por constructor.	Reglas del Plan de Trabajo y README del proyecto.
RNF2 4	El sistema cuenta con pruebas automatizadas unitarias y de integración que se pueden ejecutar con un solo comando.	72 clases de prueba con 379 casos (JUnit 5, Mockito, Spring Test y GreenMail). Se ejecutan con <i>mvnw clean verify</i> .
RNF2 5	El código fuente se versiona en un repositorio Git y las tareas se organizan en un tablero Kanban.	Repositorio en GitHub; tablero en Trello.

6.6. Portabilidad, instalación y tecnología

ID	Requerimiento	Cómo se cumple
RNF2 6	Tecnologías: Java 21, Spring Boot 3.5, Thymeleaf, Spring Data JPA con Hibernate, SQLite, Spring Security, Spring Mail y Bootstrap 4 (incluido en el template).	Dependencias declaradas en pom.xml.
RNF2 7	La instalación solo requiere el JDK 21: no hace falta instalar Maven, un servidor de aplicaciones ni un motor de base de datos. Al iniciar por primera vez, la base se crea y se carga con datos iniciales.	Maven Wrapper, Tomcat embebido (puerto 8080), SQLite embebido en <i>data/zero.db</i> y DataSeeder.
RNF2 8	El servidor de correo (SMTP) y la URL pública del sitio se configuran sin modificar el código.	Pantalla de configuración de correo de la empresa; variable de entorno <i>APP_URL_BASE</i> .

6.7. Restricciones del negocio

ID	Requerimiento	Cómo se cumple
RNF2 9	Los pagos con Mercado Pago son simulados: no se realizan cobros reales ni se usan credenciales externas. El pago lo confirma un administrador desde el panel.	Forma de pago BILLETERA_VIRTUAL sin SDK externo; confirmación en el panel de pedidos.
RNF3 0	Las facturas se numeran en forma secuencial por tipo (cliente y proveedor), simulando la numeración validada por ARCA.	<i>siguienteNumero()</i> en VentaService y FacturaProveedorService.
RNF3 1	Para no aumentar los costos de desarrollo no se contratan diseñadores: el diseño se basa en un template HTML/CSS existente, adaptado por el equipo a la marca Zero.	Template en <i>static/vendor/template</i> .

7. Patrones de software

El enunciado solicita implementar como mínimo cinco patrones. Se describen a continuación los patrones Capas, Modelo-Vista-Controlador, DAO, OID y DTO, con fragmentos del código real, y los patrones GRASP aplicados en los casos de uso críticos.

Patrón	Dónde se aplica	Clases involucradas
Capas	Toda la aplicación	controllers, services, repositories, entities
MVC	Cada pantalla del sitio y del panel	38 controladores, 82 plantillas Thymeleaf, entidades y DTO
DAO	Acceso a la base de datos y a archivos	29 repositorios Spring Data JPA y NewsletterEnvioRepository
OID	Identificación de todas las entidades	BaseEntity (id UUID) y sus 29 entidades persistentes
DTO	Reportes, dashboard, catálogo y formularios	42 clases del paquete dto

7.1. Capas

Organiza el software en niveles con una responsabilidad cada uno; cada capa usa solo a la inmediatamente inferior. En el alta de una nacionalidad, el controlador delega en el servicio, que valida y persiste mediante el repositorio:

```
// Presentación: el controlador delega en el servicio
service.crearNacionalidad(nombre);

// Negocio: el servicio valida y usa el repositorio
public void crearNacionalidad(String nombre) throws ErrorServiceException {
    validar(nombre);
    Nacionalidad nacionalidad = new Nacionalidad();
    nacionalidad.setNombre(nombre.strip());
    repository.save(nacionalidad); // Persistencia
}
```

7.2. Modelo-Vista-Controlador (MVC)

El **modelo** son las entidades, los DTO y los servicios; la **vista**, las plantillas Thymeleaf que decoran los layouts público y administrativo; el **controlador**, las clases `@Controller`. El controlador pide los datos, los agrega al modelo y devuelve el nombre de la vista:

```
@GetMapping
public String listar(Model model) {
    model.addAttribute("pageTitle", "Nacionalidades");
    model.addAttribute("nacionalidades", service.listarNacionalidadActiva());
    return "admin/nacionalidades/listado"; // vista
}
```

7.3. DAO (Data Access Object)

El acceso a datos está encapsulado en repositorios: 29 interfaces que extienden `JpaRepository` (Hibernate genera el SQL a partir del nombre de cada método) y `NewsletterEnvioRepository`, que persiste en un archivo de texto la fecha del último newsletter.

```
public interface NacionalidadRepository extends JpaRepository<Nacionalidad, String> {
    List<Nacionalidad> findByEliminadoFalseOrderByNombreAsc();
}
```

```
Optional<Nacionalidad> findByIdAndEliminadoFalse(String id);
}
```

7.4. OID (Object Identifier)

Cada objeto persistente tiene un identificador único independiente de sus datos de negocio. Se define en *BaseEntity* como un UUID generado por Hibernate y no modificable; se usa como clave primaria, en los repositorios y en las URL (por ejemplo, */admin/nacionalidades/{id}/editar*).

```
@MappedSuperclass
public abstract class BaseEntity {
    @Id
    @UuidGenerator
    @Column(length = 36, nullable = false, updatable = false)
    private String id;

    @Column(nullable = false)
    private boolean eliminado = false;    // baja lógica
}
```

7.5. DTO (Data Transfer Object)

Los DTO transportan datos entre capas sin exponer las entidades. En el reporte de ventas, el servicio arma un *ReporteVentasDTO* con los totales y el detalle, y la vista usa directamente sus campos:

```
public record ReporteVentasDTO(int cantidadCompras, int unidadesVendidas, double montoTotal,
    List<DetalleVentaDTO> detalle, List<SubtotalFormaPagoDTO> subtotalesPorFormaPago) {}

// ReporteVentasController
model.addAttribute("reporte", service.generar(desdeEfectivo, hastaEfectivo));
return "admin/reportes/ventas";
```

7.6. Patrones GRASP aplicados

Patrón	Aplicación
Creador	OrdenCompra crea sus DetalleCompra (crearDetalle) y Factura crea sus DetalleFactura (agregarDetalle).
Experto en información	OrdenCompra calcula su total y controla sus transiciones de estado; Factura calcula su total.
Polimorfismo	Factura.getSignoStock(): FacturaProveedor devuelve +1 y FacturaCliente -1; StockService registra el movimiento sin conocer el tipo de factura.
Alta cohesión / Bajo acoplamiento	Un servicio por entidad; los servicios se comunican por sus métodos públicos.
Inyección de dependencias	Por constructor en controladores, servicios y tareas programadas.

8. Casos de uso críticos

Se seleccionaron tres casos de uso críticos, encadenados entre sí: la orden de compra (carrito) se convierte en una factura al confirmarse, y el pago de esa factura descuenta el stock. Para cada uno se presentan el diagrama de caso de uso, el escenario, el diagrama de secuencia de diseño y el prototipo de pantalla. Los prototipos son capturas de la aplicación en funcionamiento.

8.1. CU01 · Generar orden de compra

8.1.1. Diagrama de caso de uso

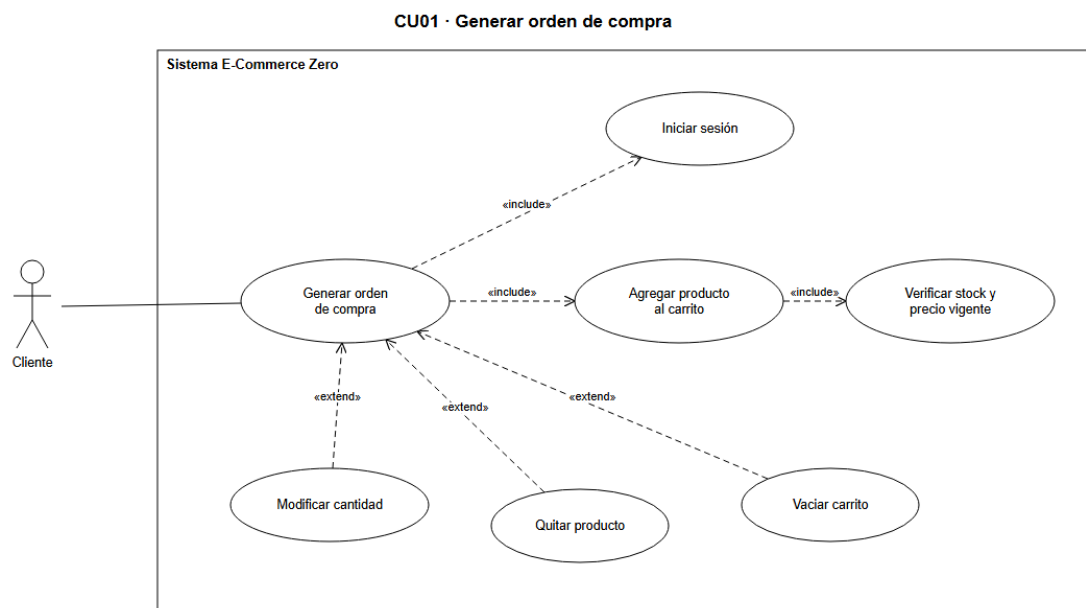


Figura 4. Diagrama de caso de uso CU01 · Generar orden de compra.

8.1.2. Escenario de caso de uso

Código	CU01
Nombre	Generar orden de compra
Actor principal	Cliente
Actores secundarios	—
Tipo	Crítico, primario
Descripción	El cliente arma su orden de compra (el carrito) agregando productos del catálogo con la cantidad deseada. La orden queda en estado PENDIENTE_COMPLETAR hasta que el cliente confirma la compra (CU02 · Generar factura).
Precondiciones	1. El cliente tiene una cuenta activa e inició sesión con el rol CLIENTE. 2. El producto está activo, tiene un precio vigente y stock disponible.
Disparador	El cliente presiona «Agregar al carrito» en el detalle de un producto.

Flujo normal	1. El cliente navega el catálogo y abre el detalle de un producto. 2. El sistema muestra la imagen, el código, el nombre, el precio vigente, el talle y las unidades disponibles. 3. El cliente indica la cantidad y presiona «Agregar al carrito». 4. El sistema valida que la cantidad sea mayor a 0 y que el producto exista, tenga precio vigente y stock disponible. 5. El sistema busca la orden de compra del cliente en estado PENDIENTE_COMPLETAR. Si no existe, crea una nueva con identificador ORD-XXXXXXX, la fecha actual y total 0. 6. El sistema verifica que la cantidad total del producto en la orden no supere el stock disponible. 7. El sistema crea el detalle de compra con el precio vigente, calcula el subtotal y recalcula el total de la orden. 8. El sistema guarda la orden y actualiza el contador del carrito. 9. El cliente abre el carrito. El sistema muestra cada producto con su imagen, talle, precio unitario, cantidad y subtotal, y el total del carrito. 10. El cliente presiona «Continuar al pago» y el caso de uso continúa en CU02 · Generar factura.
Flujos alternativos	3a. El visitante no inició sesión: el sistema lo lleva al login guardando el producto y la cantidad. Al iniciar sesión agrega el producto al carrito y vuelve al detalle del producto. 3b. El usuario es un empleado: el botón aparece deshabilitado como «Solo clientes». 4a. Cantidad menor o igual a 0: «La cantidad a agregar debe ser mayor a 0.» 4b. Producto inexistente o dado de baja: «El producto no existe o fue eliminado.» 4c. Producto sin precio vigente: «El producto no tiene un precio vigente.» 4d. Producto sin stock: «El producto "X" no cuenta con stock disponible.» 6a. El producto ya está en la orden: el sistema suma la cantidad al detalle existente y recalcula su subtotal, sin crear un renglón nuevo. 6b. La cantidad supera el stock: «No hay suficiente stock para agregar esa cantidad. Stock disponible: N (ya tenés M en tu carrito).» 9a. Modificar cantidad: el cliente cambia la cantidad de un producto; el sistema valida el stock y el precio vigente, recalcula el subtotal y el total y muestra «Cantidad actualizada.» 9b. Quitar producto: el sistema elimina el detalle, recalcula el total y muestra «Producto eliminado del carrito.» 9c. Vaciar carrito: el sistema elimina todos los detalles y muestra «Se vació el carrito.» 9d. Al abrir el carrito, si un producto se dio de baja, quedó sin precio vigente o sin stock suficiente, el sistema lo retira o ajusta su cantidad y muestra un aviso. En todos los casos de error la orden no se modifica y el sistema muestra el mensaje al cliente.
Postcondiciones	Existe una OrdenCompra del cliente en estado PENDIENTE_COMPLETAR con sus DetalleCompra, subtotales y total calculados con el precio vigente. El stock no se modifica.
Reglas de negocio	• El stock no se reserva al agregar al carrito: se vuelve a controlar al confirmar la compra. • Cada cliente tiene un solo carrito abierto, guardado en la base de datos aunque cierre sesión. • Cada talle es un producto distinto, con su propio stock.

8.1.3. Diagrama de secuencia de diseño

CU01 · Generar orden de compra: agregar producto al carrito

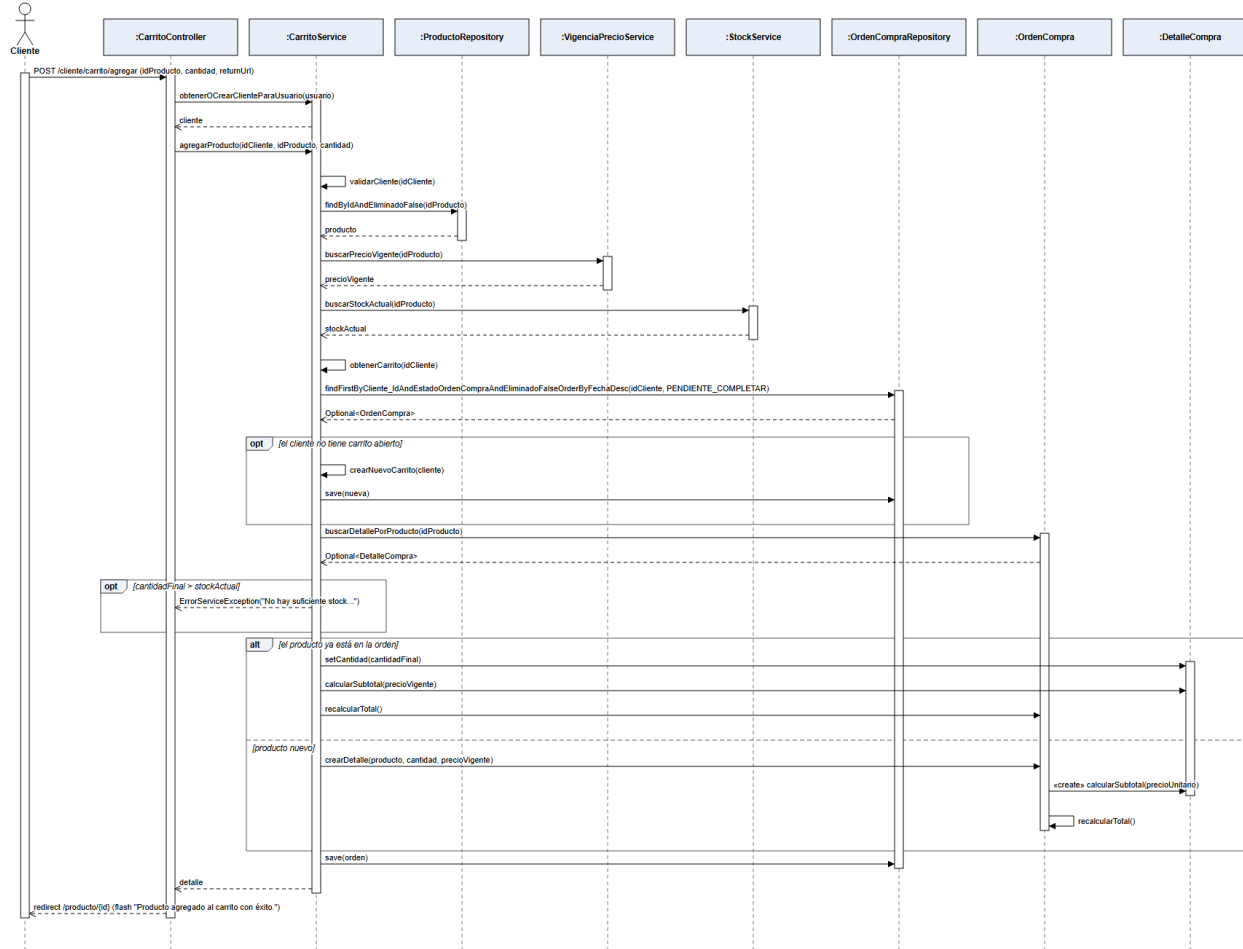


Figura 5. Diagrama de secuencia de diseño CU01 · Generar orden de compra.

8.1.4. Prototipo de pantalla

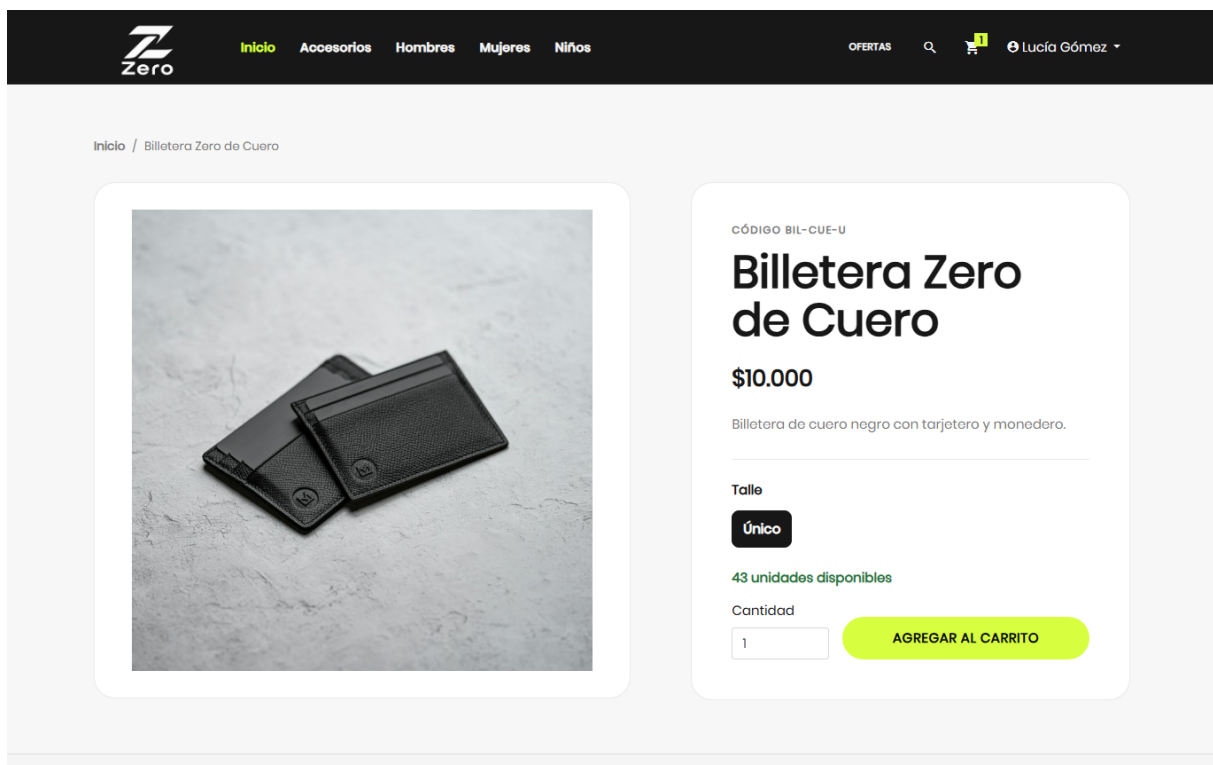


Figura 6. Detalle del producto con stock disponible.

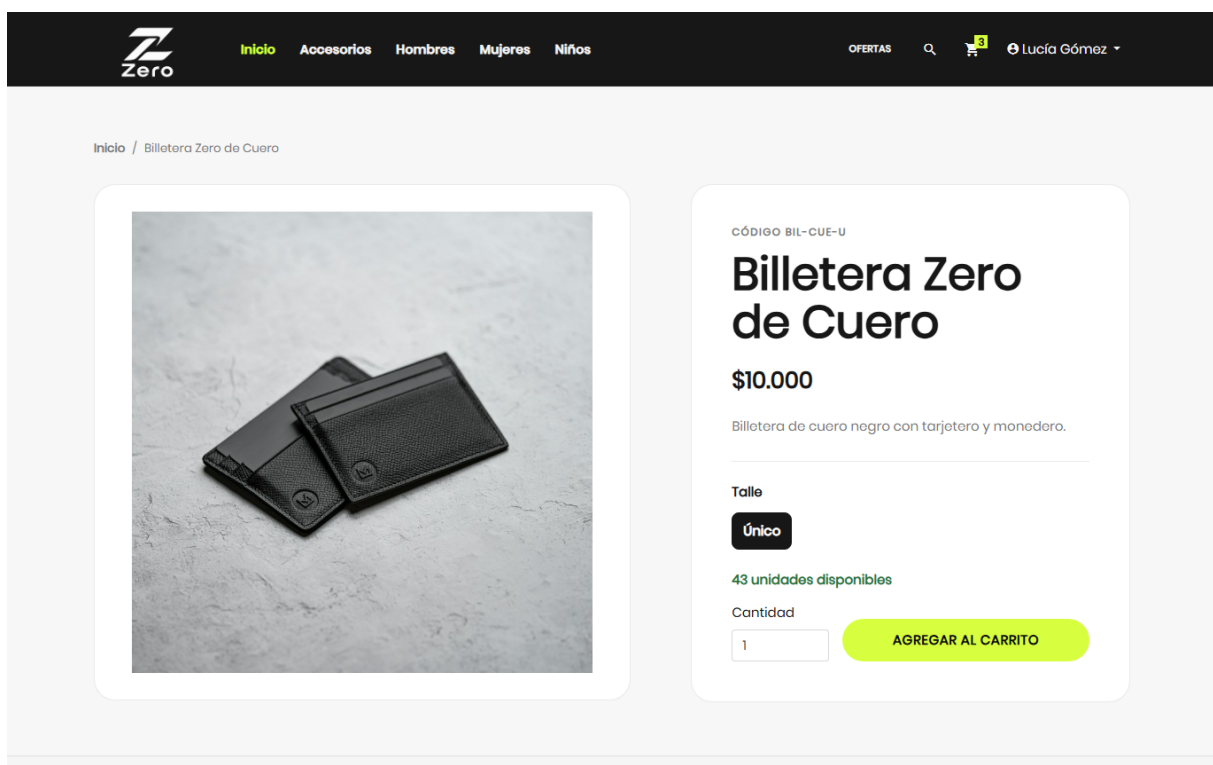


Figura 7. El contador del carrito se actualiza al agregar el producto.


[Inicio](#)
[Accesorios](#)
[Hombres](#)
[Mujeres](#)
[Niños](#)
[OFERTAS](#)


Lucía Gómez

[Inicio](#) / [Mi cuenta](#) / [Carrito de compras](#)

MI COMPRA

Carrito de compras

Imagen	Producto	Talle	Precio unitario	Cantidad	Subtotal	Acción
	Gorra Zero Training Código: GOR-TRN-U	Único	\$10.000	1	\$10.000	
	Billetera Zero de Cuero Código: BIL-CUE-U	Único	\$10.000	2	\$20.000	

 Vaciar carrito
  Seguir comprando

Total del carrito: **\$30.000**
[CONTINUAR AL PAGO](#)



ZERO

Ropa y accesorios deportivos para moverte con energía, estilo y presencia.

NAVEGACIÓN

- [Inicio](#)
- [Hombres](#)
- [Mujeres](#)
- [Ofertas](#)

ATENCIÓN

- [Soporte](#)
- [Envíos](#)
- [Cambios](#)
- [Preguntas frecuentes](#)

CONTACTO

- +54 11 5555-0101
- hola@zero.com.ar
- Av. Corrientes 1234, CABA

Figura 8. Carrito de compras con subtotales y total.

8.2. CU02 · Generar factura

8.2.1. Diagrama de caso de uso

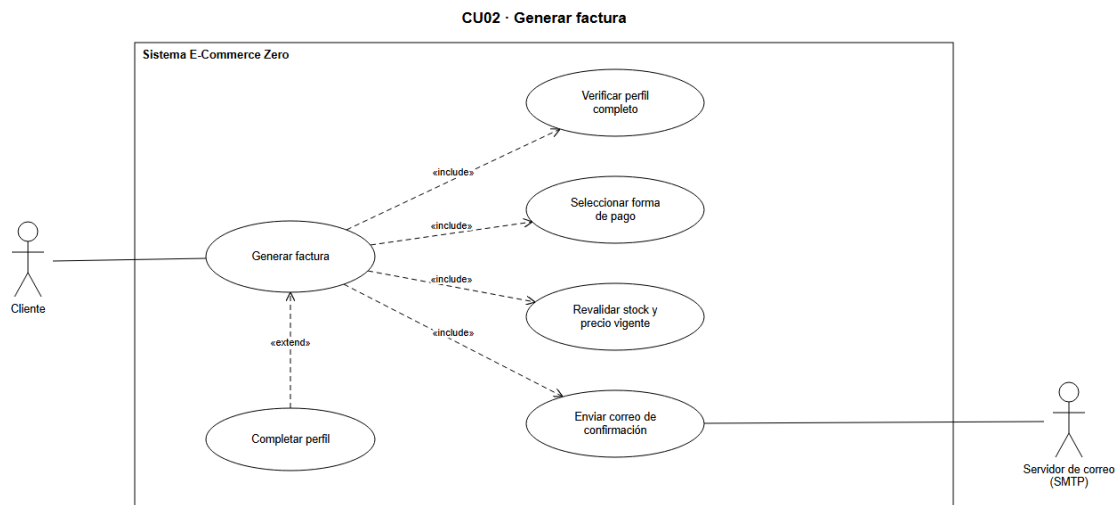


Figura 9. Diagrama de caso de uso CU02 · Generar factura.

8.2.2. Escenario de caso de uso

Código	CU02
Nombre	Generar factura
Actor principal	Cliente
Actores secundarios	Servidor de correo (SMTP)
Tipo	Crítico, primario
Descripción	El cliente confirma su orden de compra eligiendo una forma de pago. El sistema vuelve a controlar precios y stock, confirma la orden y genera la factura del cliente con sus detalles, número correlativo y total. Luego le envía un correo con el detalle de la compra.
Precondiciones	1. El cliente inició sesión con el rol CLIENTE. 2. El cliente tiene una orden de compra en estado PENDIENTE_COMPLETAR con al menos un producto (CU01). 3. El perfil del cliente está completo: datos personales, dirección de entrega y teléfono. 4. Existe al menos una forma de pago activa.
Disparador	El cliente presiona «Continuar al pago» en el carrito.

Flujo normal	1. El sistema verifica que el perfil del cliente esté completo. 2. El sistema revisa los productos de la orden (bajas, precio vigente y stock) y muestra el resumen del pedido, la dirección de entrega y las formas de pago activas. 3. El cliente elige la forma de pago (Efectivo, Transferencia o Mercado Pago) y presiona «Confirmar compra». 4. El sistema valida la forma de pago. 5. Por cada producto de la orden, el sistema controla que siga activo, con precio vigente y stock suficiente, y recalcula el subtotal con el precio vigente y el total de la orden. 6. El sistema confirma la orden de compra: pasa de PENDIENTE_COMPLETAR a PENDIENTE_PAGO y registra la fecha. 7. El sistema genera la factura del cliente con el número siguiente al último emitido, la fecha actual, el estado SIN_DEFINIR, el cliente, la orden de compra y la forma de pago. 8. Por cada producto, el sistema agrega un detalle de factura con la cantidad y el precio vigente, y calcula el total de la factura. 9. El sistema guarda la factura y le envía al cliente, en segundo plano, el correo «Recibimos tu compra» con el detalle, el número de factura y las instrucciones de pago. 10. El sistema muestra «Compra registrada» con el número de compra, el estado, el total, la forma de pago, el número de factura y cómo pagar.
Flujos alternativos	1a. Perfil incompleto: «Completá tu perfil (datos, dirección de entrega y teléfono) para poder confirmar la compra.» El sistema lleva al cliente a su perfil (extensión Completar perfil). 2a. La orden está vacía: «Tu carrito está vacío.» y vuelve al carrito. 2b. Hubo ajustes (producto dado de baja, sin precio o sin stock suficiente): el sistema actualiza la orden y vuelve al carrito mostrando los avisos. 4a. No eligió forma de pago: «Elegí una forma de pago.» 4b. La forma de pago fue dada de baja: «La forma de pago no existe o fue eliminada.» 5a. Un producto fue dado de baja: «Un producto de tu carrito ya no está disponible. Revisá tu carrito antes de confirmar.» 5b. Un producto no tiene precio vigente: «El producto “X” no tiene precio vigente. Revisá tu carrito antes de confirmar.» 5c. Stock insuficiente: «No hay stock suficiente de “X” (talle T). Stock disponible: N. Revisá tu carrito antes de confirmar.» En 4a a 5c la operación se revierte por completo (no se confirma la orden ni se genera la factura) y el cliente vuelve a la pantalla de confirmación con el mensaje. 9a. Falla el envío del correo: el error queda registrado en el log y la compra se mantiene.
Postcondiciones	1. La OrdenCompra queda en estado PENDIENTE_PAGO. 2. Existe una FacturaCliente en estado SIN_DEFINIR con sus DetalleFactura, número correlativo y total. 3. Se envió el correo de confirmación al cliente. 4. El stock todavía no se descuenta: baja cuando un empleado confirma el pago (CU03 · Actualizar stock).
Reglas de negocio	• Cada detalle de factura guarda el precio vigente al confirmar: un cambio de precio posterior no altera la venta. • La numeración de facturas es correlativa por tipo de factura (simula la numeración validada por ARCA). • Mercado Pago es simulado: no se realiza ningún cobro real y el pago lo confirma un empleado.

8.2.3. Diagrama de secuencia de diseño

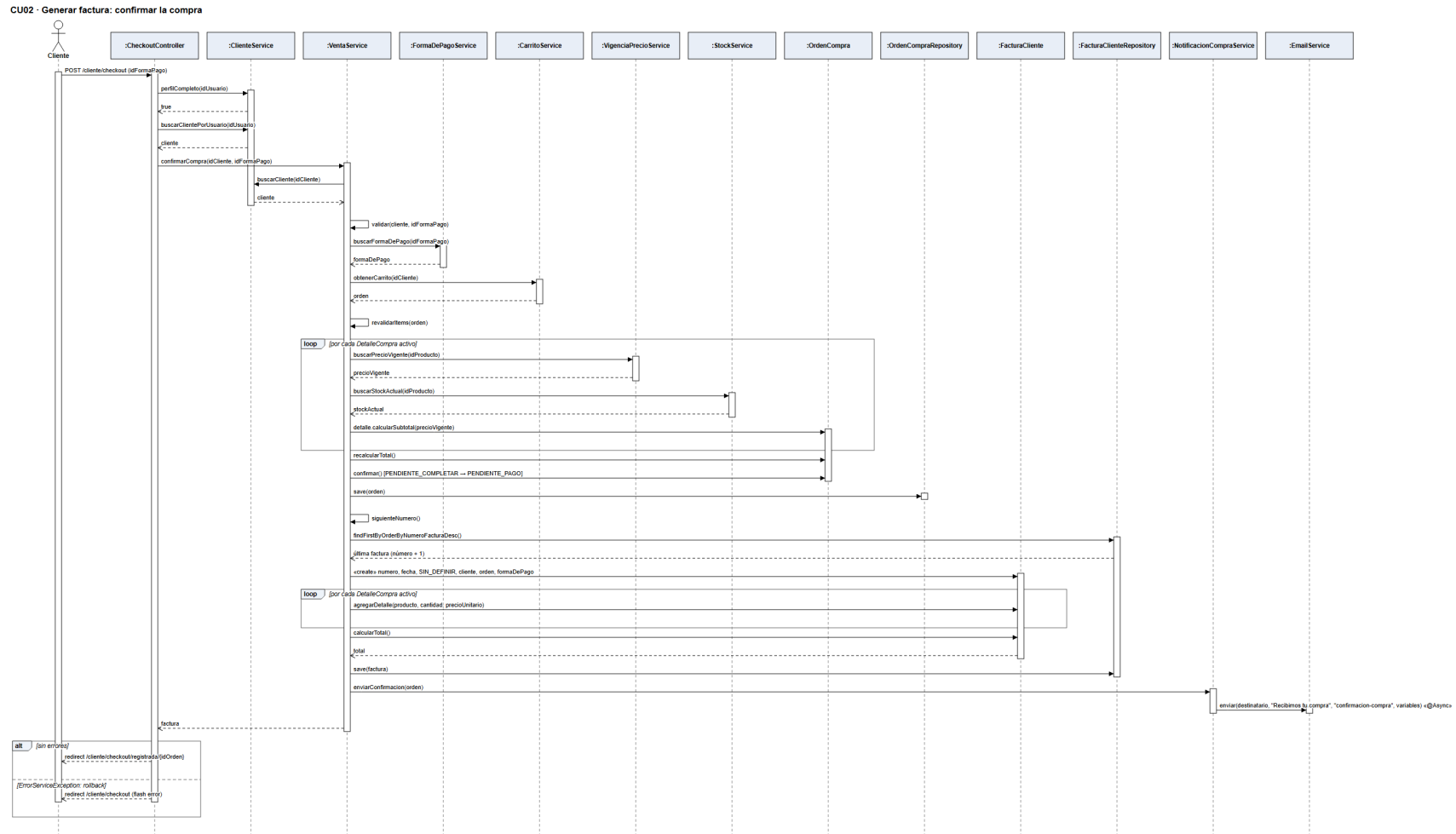


Figura 10. Diagrama de secuencia de diseño CU02 · Generar factura.

8.2.4. Prototipo de pantalla

Inicio / Carrito de compras / Confirmar compra

MI COMPRA

Confirmar compra

Resumen del pedido

Producto	Talle	Cantidad	Precio	Subtotal
Oorra Zero Training	Único	1	\$10.000,00	\$10.000,00
Billotera Zero de Cuero	Único	2	\$10.000,00	\$20.000,00
Total				\$30.000,00

Dirección de entrega

San Martín 1250, Ciudad de Mendoza (5500), Mendoza

MODIFICAR DIRECCIÓN

Forma de pago

Eligi cómo pagar

Mercado Pago (Billettera Virtual)

CONFIRMAR COMPRA

VOLVER AL CARRITO

ZERO

Ropa y accesorios deportivos para moverte con energía, estilo y presencia.

NAVEGACIÓN

- Inicio
- Hombres
- Mujeres
- Ofertas

ATENCIÓN

- Soporte
- Envíos
- Cambios
- Preguntas frecuentes

CONTACTO

+54 11 5555-0101

hola@zero.com.ar

Av. Corrientes 1234, CABA

Figura 11. Confirmación de la compra: resumen, dirección de entrega y forma de pago.

Inicio / Mi cuenta / Compra registrada

¡GRACIAS POR TU COMPRA!

Compra registrada

Tu número de compra es ORD-DEMO0006

Te enviamos un correo con el detalle de la compra.

Estado

Pendiente de pago

Total

\$30.000,00

Forma de pago

Mercado Pago

Factura

N° 15

Cómo pagar

Mercado Pago simulado: el pago está pendiente de confirmación administrativa. No se realiza ningún cobro real.

SEGUIR MI COMPRA

SEGUIR COMPRANDO

ZERO

Ropa y accesorios deportivos para moverte con energía, estilo y presencia.

NAVEGACIÓN

- Inicio
- Hombres
- Mujeres
- Ofertas

ATENCIÓN

- Soporte
- Envíos
- Cambios
- Preguntas frecuentes

CONTACTO

+54 11 5555-0101

hola@zero.com.ar

Av. Corrientes 1234, CABA

Figura 12. Compra registrada con el número de factura.

8.3. CU03 · Actualizar stock

8.3.1. Diagrama de caso de uso

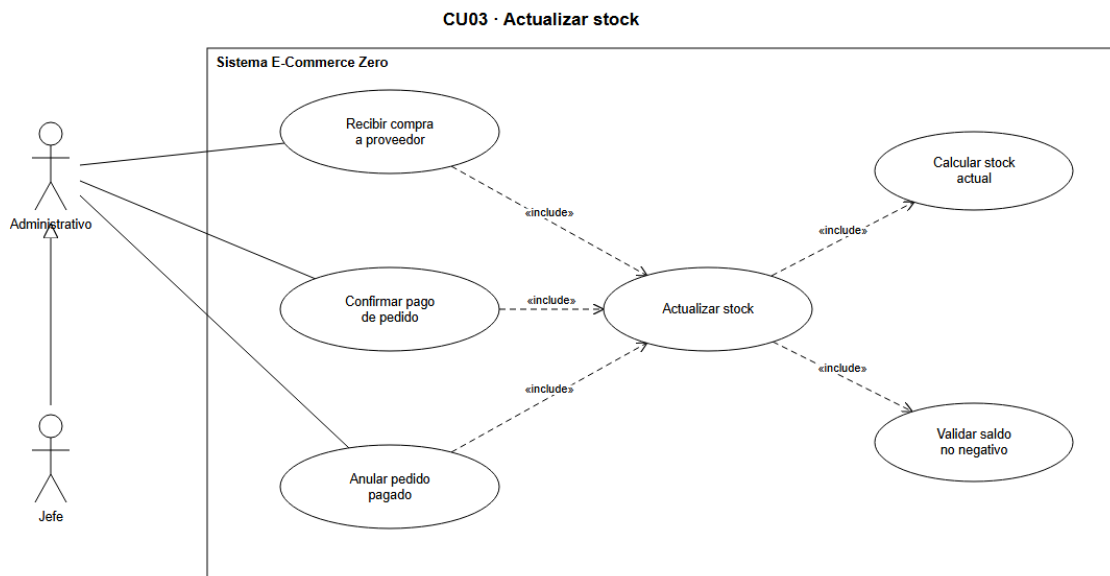


Figura 13. Diagrama de caso de uso CU03 · Actualizar stock.

8.3.2. Escenario de caso de uso

Código	CU03
Nombre	Actualizar stock
Actor principal	Administrativo (el Jefe hereda sus permisos)
Actores secundarios	—
Tipo	Crítico, primario
Descripción	El stock de cada producto se actualiza registrando movimientos. Aumenta cuando un empleado marca como recibida una compra a proveedor y disminuye cuando confirma el pago de una venta. El escenario principal describe la recepción de mercadería.
Precondiciones	1. El empleado inició sesión con el rol ADMINISTRATIVO o JEFE. 2. Existe una compra a proveedor en estado Pedida (factura del proveedor SIN_DEFINIR) con al menos un producto.
Disparador	Llega la mercadería del proveedor y el empleado la registra en el sistema.

Flujo normal	1. El empleado ingresa a «Compras a proveedor» y abre una compra en estado Pedida. 2. El sistema muestra los datos de la compra (número, fecha, proveedor, forma de pago y estado) y sus productos con cantidad, precio de costo y subtotal. 3. El empleado presiona «Marcar como recibida» y confirma en el diálogo. 4. El sistema busca la compra y verifica que siga en estado Pedida. 5. El sistema cambia el estado de la factura del proveedor a PAGADA (recibida). 6. Por cada producto de la compra, el sistema registra un movimiento de stock: 6.1. Obtiene el signo del movimiento según el tipo de factura: +1 para la compra a proveedor. 6.2. Obtiene el stock actual del producto (el saldo de su último movimiento) y calcula el nuevo saldo sumándole la cantidad recibida. 6.3. Verifica que el nuevo saldo no sea negativo. 6.4. Guarda el movimiento (Stock) con el producto, el detalle de factura, el nuevo saldo, la fecha y la observación «Compra N.º X». 7. El sistema guarda la compra y muestra «Compra N.º X recibida: se sumó la mercadería al stock.» 8. El stock actual de cada producto pasa a ser el nuevo saldo y se ve en el detalle del producto (historial de movimientos) y en el reporte de stock.
Flujos alternativos	1a. Confirmar pago de una venta (disminuye el stock): el empleado abre un pedido en estado PENDIENTE_PAGO y presiona «Confirmar pago». El sistema asocia el empleado a la factura, pasa la orden a PENDIENTE_ENVIO y la factura a PAGADA, y registra por cada producto un movimiento con signo -1 y la observación «Venta N.º X». Muestra «Se confirmó el pago del pedido ORD-...». 1b. Anular un pedido pagado (revierte el stock): el empleado anula un pedido en PENDIENTE_ENVIO indicando el motivo. El sistema anula la orden y la factura y registra por cada producto un movimiento inverso con la observación «Anulación», que devuelve las unidades al stock. 3a. El empleado cancela el diálogo: no se modifica nada. 4a. La compra ya fue recibida: «La compra N.º X ya fue recibida: no se puede recibir de nuevo.» 4b. La compra está anulada: «La compra N.º X está anulada: no se puede recibir.» 6a. La cantidad de un movimiento no es mayor a 0: «La cantidad del movimiento de stock tiene que ser mayor a 0.» 6.3a. El nuevo saldo sería negativo (al confirmar el pago de una venta): «No hay stock suficiente de X (talle T): hay N y se necesitan M.» Ante cualquier error la operación se revierte completa: no se guarda ningún movimiento ni el cambio de estado, y el sistema muestra el mensaje.
Postcondiciones	1. La compra queda en estado Recibida (PAGADA) y ya no se puede anular. 2. Por cada producto existe un nuevo movimiento de stock y su stock actual es el saldo de ese movimiento.
Reglas de negocio	• El stock se guarda como movimientos: el stock actual es el saldo del último movimiento del producto. • El signo del movimiento lo decide la factura por polimorfismo (Factura.getSignoStock()): FacturaProveedor devuelve +1 y FacturaCliente devuelve -1. • El stock nunca puede quedar negativo.

8.3.3. Diagrama de secuencia de diseño

CU03 · Actualizar stock: recibir compra a proveedor

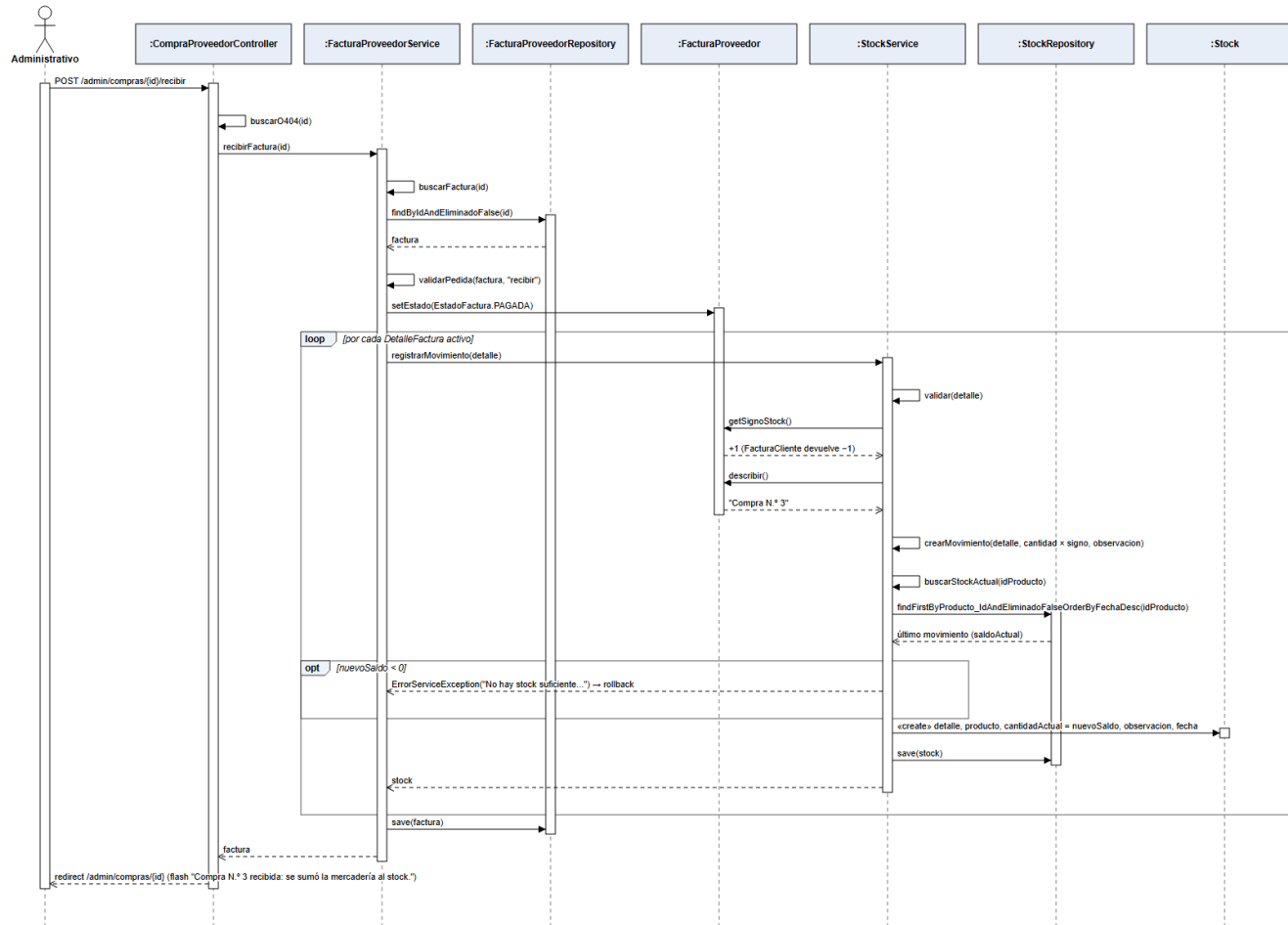


Figura 14. Diagrama de secuencia de diseño CU03 · Actualizar stock.

8.3.4. Prototipo de pantalla

- Inicio
- Productos**
- Precios 20
- Actualización por inflación
- Alertas de precios
- Categorías
- Proveedores
- Compras a proveedor
- Pedidos
- Newsletter
- REPORTES
- Ventas
- Stock
- Proveedores

PANEL ADMINISTRATIVO

Administrativo Zero ADMINISTRATIVO Cerrar sesión

Catálogo / Productos / Detalle

Remera Zero Dry Fit Hombre (talle M)

Datos

Código
REM-DRY-H-M

Nombre
Remera Zero Dry Fit Hombre

Talle
M

Subcategoría
Hombres / Ropa

STOCK ACTUAL

19 unidades

Saldo del último movimiento

EDITAR

VOLVER

Historial de movimientos

Del más reciente al más antiguo. Las compras recibidas suman y las ventas restan.

Fecha	Movimiento	Cantidad	Saldo
28/07/2026 13:26	Stock inicial de demostracion	+19	19

Figura 15. Historial de stock del producto antes de recibir la compra.

- Inicio
- Productos
- Precios 20
- Actualización por inflación
- Alertas de precios
- Categorías
- Proveedores
- Compras a proveedor**
- Pedidos
- Newsletter
- REPORTES
- Ventas
- Stock
- Proveedores

PANEL ADMINISTRATIVO

Administrativo Zero ADMINISTRATIVO Cerrar sesión

Abastecimiento / Compras a proveedor / Detalle

Compra N.º 3

Datos

Número
3

Fecha
28/09/2026

Proveedor
Distribuidora Deportiva Cuyo S.A.

Forma de pago
Transferencia

Estado
Sin definir Pedida: todavía no se recibió la mercadería.

Avisar al proveedor por WhatsApp

MARCAR COMO RECIBIDA

Anular compra

VOLVER

Productos

Código	Producto	Talle	Cantidad	Precio de costo	Subtotal
REM-DRY-H-M	Remera Zero Dry Fit Hombre	M	20	\$4.500,00	\$90.000,00
REM-DRY-H-L	Remera Zero Dry Fit Hombre	L	15	\$4.500,00	\$67.500,00
SHO-RUN-H-M	Short Zero Run Hombre	M	10	\$3.800,00	\$38.000,00
Total					\$195.500,00

Figura 16. Detalle de una compra a proveedor en estado Pedida.

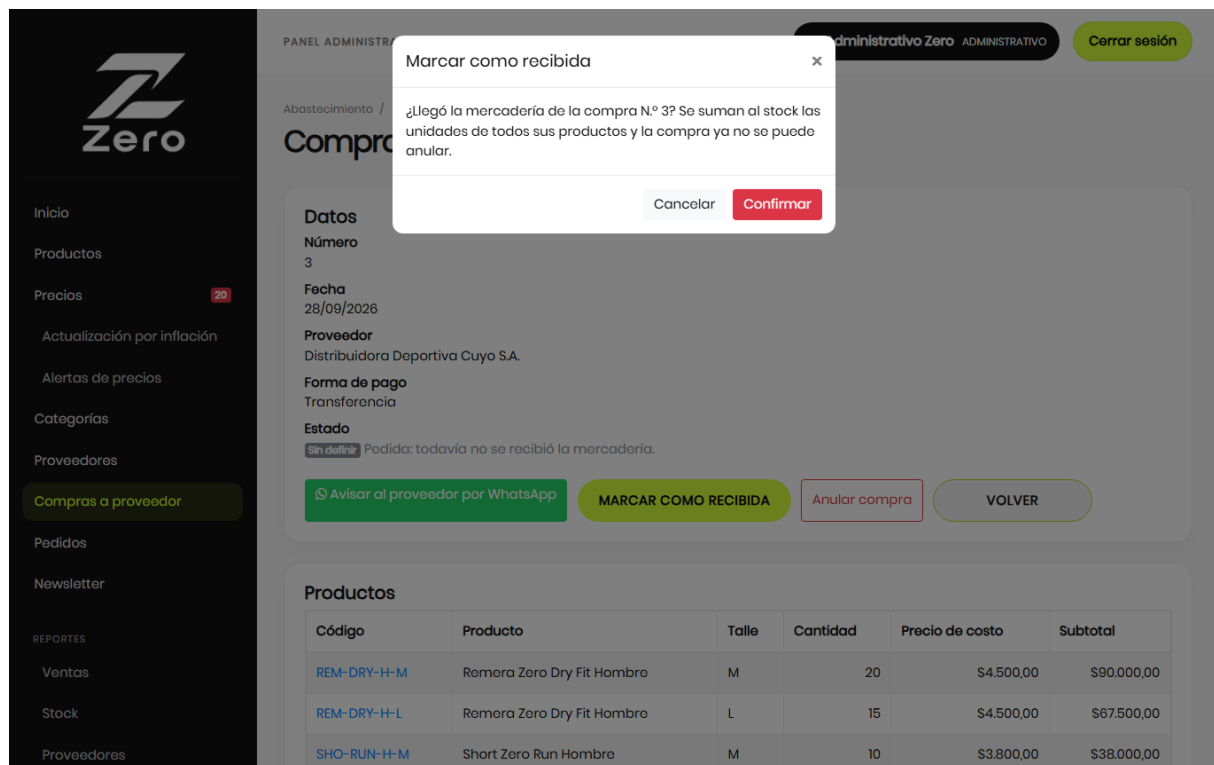


Figura 17. Confirmación de la recepción de la mercadería.

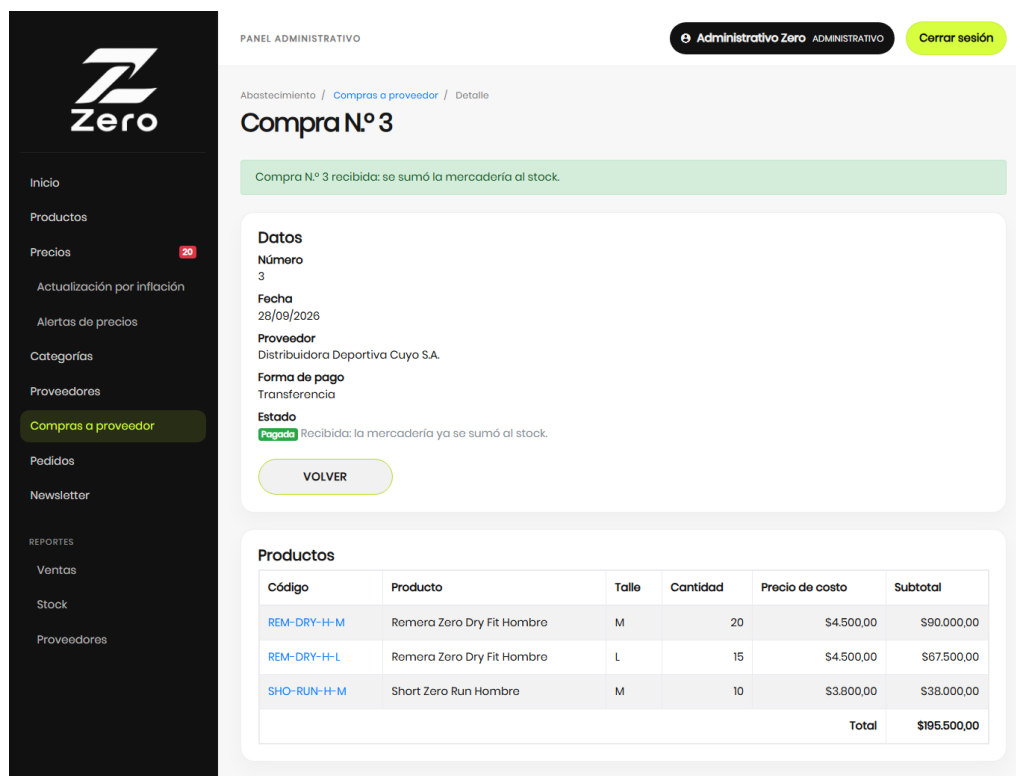
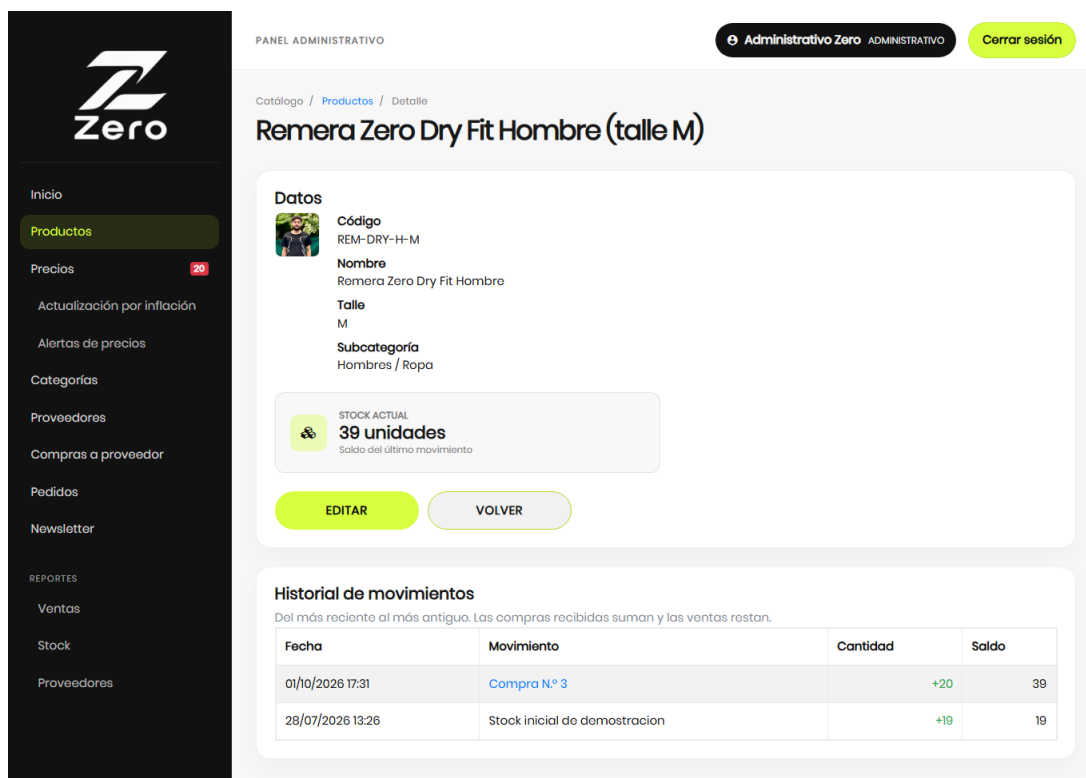


Figura 18. Compra recibida: se sumó la mercadería al stock.



PANEL ADMINISTRATIVO **Administrativo Zero** ADMINISTRATIVO **Cerrar sesión**

Catálogo / Productos / Detalle

Remera Zero Dry Fit Hombre (talle M)

Datos

Código
REM-DRY-H-M

Nombre
Remera Zero Dry Fit Hombre

Talle
M

Subcategoría
Hombres / Ropa

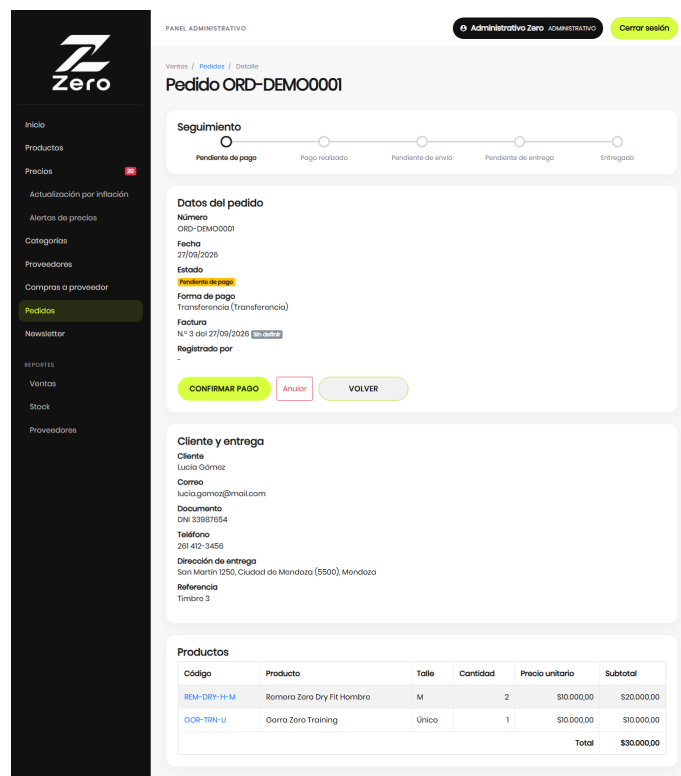
STOCK ACTUAL
39 unidades
Saldo del último movimiento

EDITAR **VOLVER**

Historial de movimientos
Del más reciente al más antiguo. Las compras recibidas suman y las ventas restan.

Fecha	Movimiento	Cantidad	Saldo
01/10/2026 17:31	Compra N° 3	+20	39
28/07/2026 13:26	Stock inicial de demostración	+19	19

Figura 19. Historial actualizado: el stock pasa de 19 a 39 unidades.



PANEL ADMINISTRATIVO **Administrativo Zero** ADMINISTRATIVO **Cerrar sesión**

Ventas / Pedidos / Detalle

Pedido ORD-DEMO0001

Seguimiento

Pendiente de pago → Pago recibido → Pendiente de envío → Pendiente de entrega → Entregado

Datos del pedido

Número
ORD-DEMO0001

Fecha
27/09/2026

Estado
Pendiente de pago

Forma de pago
Transferencia (Transferencia)

Factura
N° 3 del 27/09/2026 [Ver factura](#)

Registrado por
-

CONFIRMAR PAGO **Anular** **VOLVER**

Cliente y entrega

Cliente
Lucia Gomez

Correo
lucia.gomez@mail.com

Documento
DNI 33987654

Teléfono
261 402-3456

Dirección de entrega
San Martín 1250, Ciudad de Mendoza (5500), Mendoza

Referencia
Timbre 3

Productos

Código	Producto	Talle	Cantidad	Precio unitario	Subtotal
REM-DRY-H-M	Remera Zero Dry Fit Hombre	M	2	\$10.000,00	\$20.000,00
GOR-TRN-U	Gorra Zero Training	Único	1	\$10.000,00	\$10.000,00
Total					\$30.000,00

Figura 20. Pedido pendiente de pago: al confirmar el pago el stock disminuye.

9. Propuesta de mejora

9.1. Investigación de mercado

Se analizaron las plataformas de comercio electrónico y de gestión utilizadas por tiendas como Zero, observando cómo organizan dos tareas diarias: recibir la mercadería de los proveedores y despachar los paquetes de los clientes.

Plataforma	Qué ofrece	Limitación para Zero
Shopify	Órdenes de compra a proveedores con inventario entrante y fecha estimada de llegada, filtrables por fecha.	La información se presenta en listados y filtros, sin una vista de calendario que una llegadas y envíos.
Odoo (Inventario y Compras)	Fecha programada de cada recepción y entrega, calculada con los plazos de proveedor y de despacho.	ERP completo y costoso de implementar; la vista de calendario de recepciones requiere módulos adicionales.
Tiendanube	Estados de venta como «Por enviar» y días adicionales de preparación configurables.	No registra la fecha esperada de llegada de las compras a proveedores ni ofrece un calendario operativo.
Zero (sistema actual)	Listados de compras a proveedor y de pedidos con filtros por estado y fechas.	No guarda cuándo se espera la mercadería ni la fecha límite de despacho de cada pedido.

Las herramientas analizadas manejan ambas fechas, pero en listados separados o mediante módulos adicionales. Para una tienda pequeña, donde las mismas personas reciben mercadería y arman paquetes, resulta útil verlas en una sola pantalla y por día.

9.2. Funcionalidad propuesta: calendario de llegadas y envíos

Se propone agregar al panel administrativo un calendario mensual que muestre:

- **Llegadas de proveedores:** las compras pedidas, en la fecha estimada de entrega que se carga al crear la compra.
- **Envíos a clientes:** los pedidos pagados pendientes de envío, en su fecha límite de despacho, calculada como la fecha de pago más los días de despacho configurados en la empresa.
- **Atrasados:** compras y pedidos cuya fecha ya venció y siguen pendientes.

Cada evento enlaza con la compra o el pedido correspondiente, donde se encuentran las acciones existentes (marcar como recibida, marcar como enviado, avisar al proveedor por WhatsApp); las fechas pueden reprogramarse. El menú lateral muestra la cantidad de eventos del día. La funcionalidad está disponible para los roles Jefe y Administrativo.

9.3. Cambios en el diseño

Clase	Tipo de cambio	Detalle
CalendarioController, CalendarioAdvice	Nuevas	Pantalla /admin/calendario y contador de eventos del día en el menú.
CalendarioService	Nueva	Reúne compras y pedidos, arma el mes por semanas y detecta atrasos.

Clase	Tipo de cambio	Detalle
EventoAgendable	Nueva (interfaz)	Contrato común de lo que aparece en el calendario; la implementan FacturaProveedor y OrdenCompra.
TipoEventoCalendario	Nueva (enumeración)	LLEGADA_PROVEEDOR y ENVIO_CLIENTE.
CalendarioMesDTO, DiaCalendarioDTO, EventoCalendarioDTO	Nuevas (DTO)	Datos que se envían a la vista.
FacturaProveedor, OrdenCompra, Empresa	Modificadas	Atributos fechaEntregaEstimada, fechaLimiteEnvio y diasDespacho.
Servicios y repositorios de compras y pedidos	Modificados	Validan las fechas nuevas y consultan eventos por rango de fechas.

El diseño aplica los patrones Capas/MVC, Polimorfismo (el servicio trata compras y pedidos a través de *EventoAgendable*), Experto (cada entidad sabe si está atrasada) y DTO.

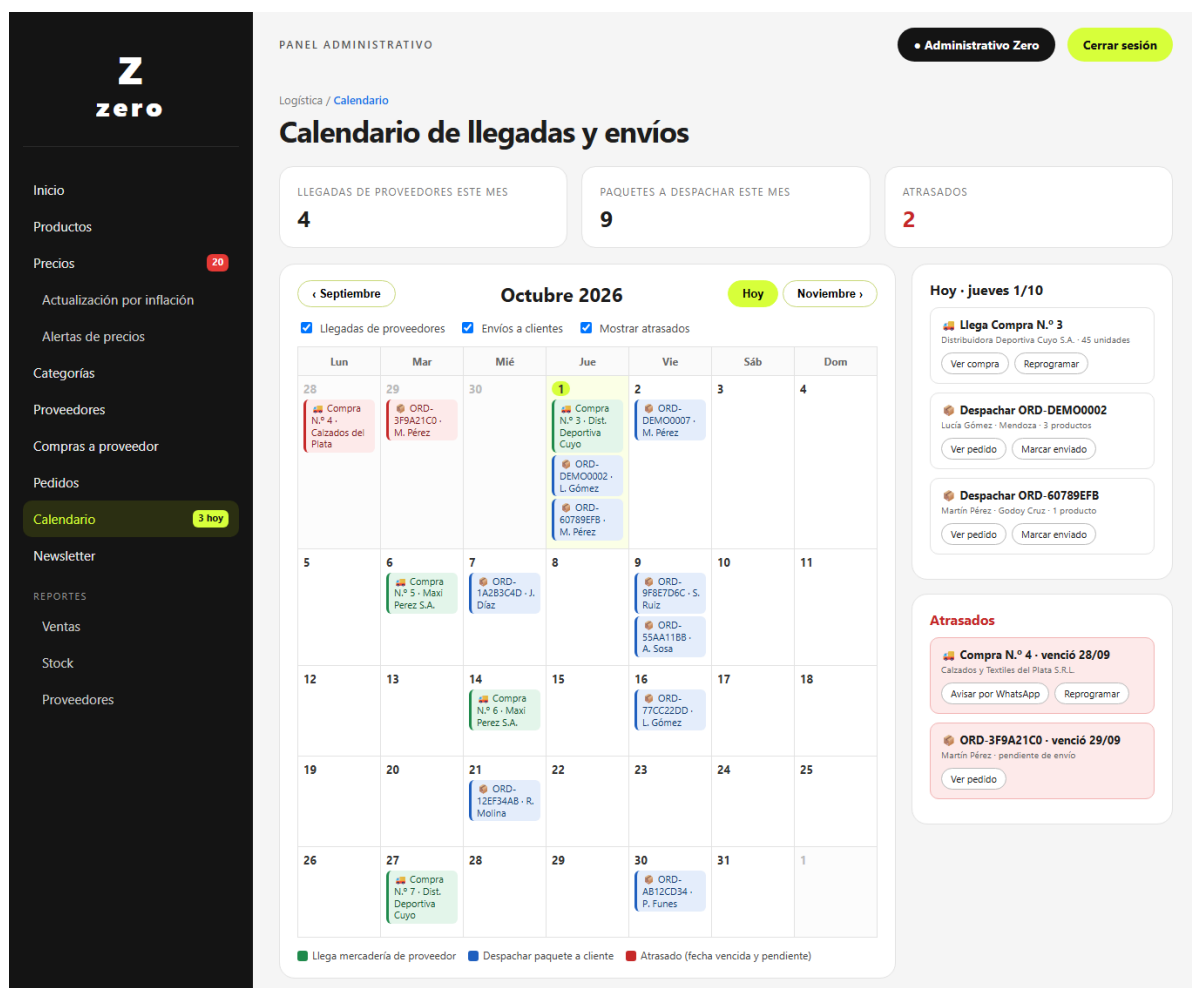


Figura 21. Prototipo del calendario en el panel administrativo (datos de ejemplo).

9.4. Diagrama de clases de diseño de la propuesta

Propuesta de mejora · Calendario de llegadas y envíos · Diagrama de clases de diseño

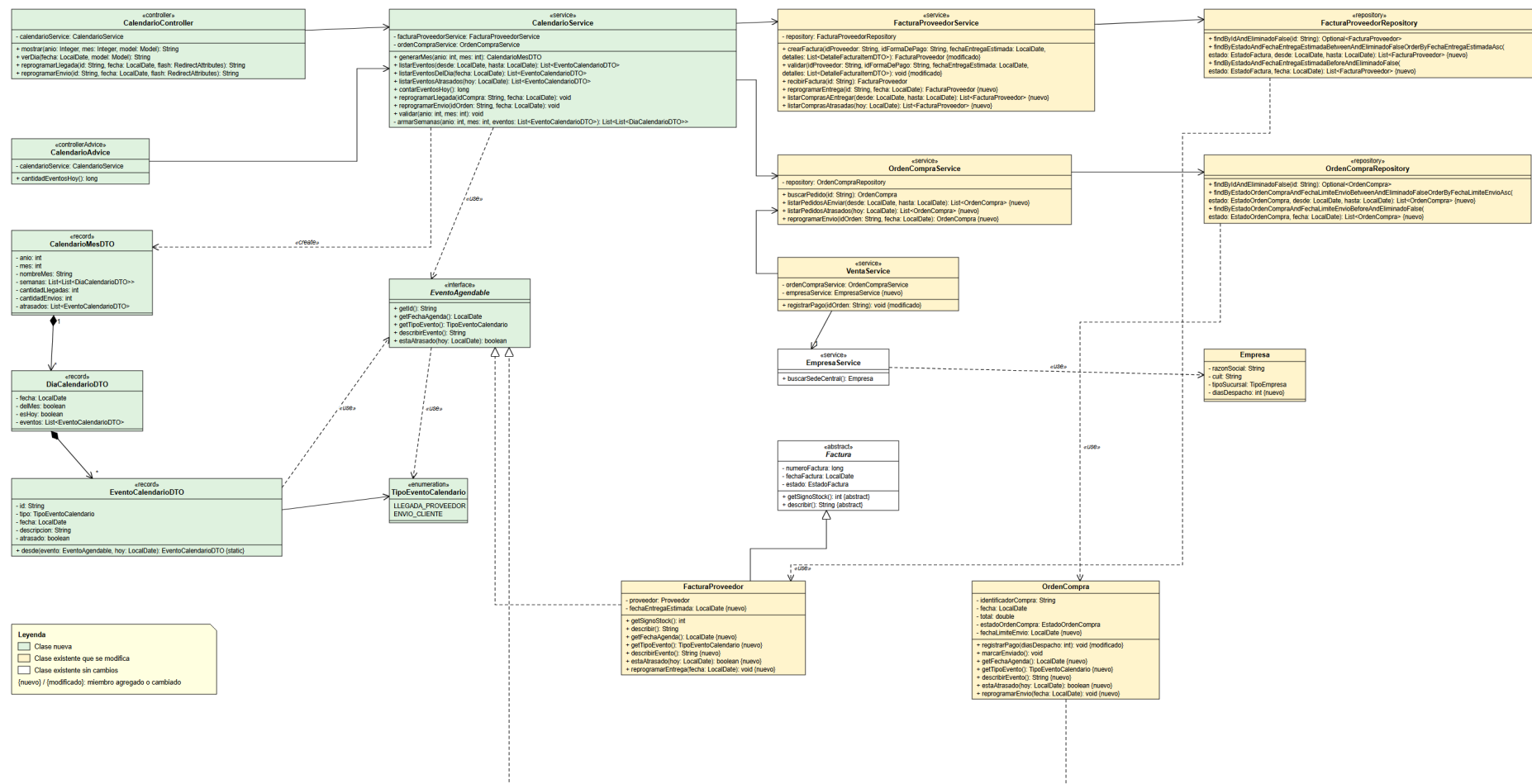


Figura 22. Diagrama de clases de diseño del calendario de llegadas y envíos. Verde: clases nuevas; amarillo: clases modificadas.

10. Conclusiones

Se desarrolló un sistema de comercio electrónico web con arquitectura Cliente-Servidor de tres capas, que cubre la venta en línea, la administración de productos, precios, proveedores y stock, y los reportes solicitados por la tienda Zero. El diseño aplica los patrones Capas, MVC, DAO, OID y DTO, además de los patrones GRASP Creador, Experto, Polimorfismo e Inyección de dependencias.

La documentación se verificó contra el código fuente: los diagramas de paquetes, entidad-relación y de secuencia reflejan las clases, métodos y relaciones reales, y los prototipos corresponden a la aplicación en funcionamiento. Como trabajo futuro se propone el calendario de llegadas y envíos descripto en la sección 9.

Anexo A. Ubicación de los entregables

Los archivos fuente de los diagramas (formato UMLet) y los documentos se encuentran en el repositorio Git del proyecto, dentro de *IntegradorN1/docs*:

Entregable	Ubicación
Diagrama de clases de diseño	docs/DiagramaUML.uxf
Casos de uso críticos	docs/Presentacion/CU01-GenerarOrdenDeCompra, CU02-GenerarFactura y CU03-ActualizarStock
Diagrama de paquetes	docs/Presentacion/DocsPresentacion/Diagrama Paquetes
Diagrama entidad-relación	docs/Presentacion/DocsPresentacion/DER
Requerimientos no funcionales	docs/Presentacion/DocsPresentacion/RNF
Patrones de software	docs/Presentacion/DocsPresentacion/Patrones
Propuesta de mejora	docs/Presentacion/DocsPresentacion/Propuesta Mejora
Presentación	docs/Presentacion/DocsPresentacion